

Inference Optimisation for Large Models

Serving, Distillation, Pruning, Quantization

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية

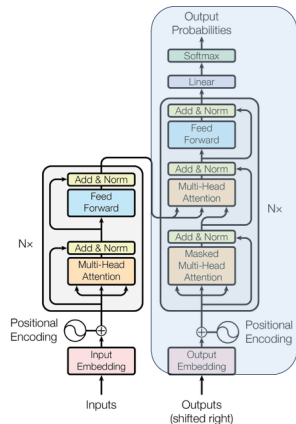
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 3, 2026

1. LLM inference: autoregressive decoding, prefill vs. decode, latency and throughput
2. Serving optimisations: KV caching, continuous batching, PagedAttention, and speculative decoding
3. Knowledge distillation: compression motivation and teacher–student architecture
4. Distillation training: soft targets, temperature, loss, and reasoning traces
5. Pruning: magnitude pruning, structured vs. unstructured, and 2:4 sparsity
6. Quantization: formats, granularity, weights/activations/KV cache, QAT and PTQ
7. Summary and references

- ▶ Most popular decoder-only LLMs (e.g., GPT-3) are pretrained for **causal language modeling**: predicting the next token in a sequence.
- ▶ At inference, LLMs receive a series of **tokens** as input and generate the following tokens **autoregressively** until a stopping criterion is met (e.g., token limit, stop word, or special <end> token).
- ▶ **Tokens** are about 4 English characters each.
- ▶ The inference process has **two phases**: **prefill** (process prompt in parallel) and **decode** (step-by-step token generation).



Highlighted: the stack a decoder-only LLM keeps. Vaswani et al., "Attention Is All You Need," [arXiv:1706.03762](https://arxiv.org/abs/1706.03762),

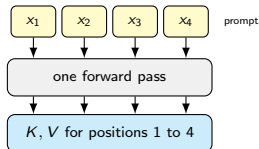
Figure 1.

► Prefill phase (input processing):

- All input tokens are processed together to compute **keys** and **values** (attention states) for the prompt.
- The entire input sequence is known in advance, enabling a highly parallel **matrix-matrix** computation that fully uses the GPU's throughput.

► Token throughput caveat:

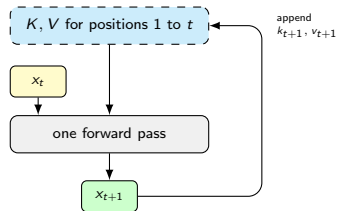
- Different LLMs may use different **tokenizers**, so “tokens per second” is not always comparable across models, as token-to-character mappings vary.



All prompt positions are computed in a single pass.

► Decode phase (generating outputs):

- Output tokens are generated **one at a time**, each conditioned on all previously generated tokens.
- Computation now becomes a **matrix-vector** operation, less parallelizable and typically **memory-bound**: moving weights/activations dominates speed, not raw compute.
- Optimising decode is crucial: efficient attention, key/value caching, and memory management are ongoing research topics.

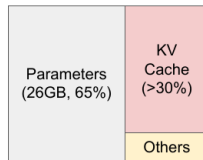


One position per pass; the cache grows every step.

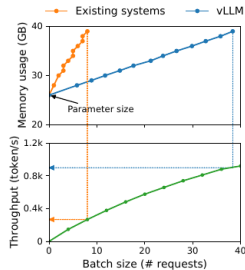
- ▶ **Training** is often one-off; **inference** is repeated at scale (cost, latency, energy).
- ▶ Large language models are **memory-bound** and **autoregressive**: each new token revisits all previous tokens.
- ▶ Goals: lower **latency** (interactive UX), higher **throughput** (serving many users), smaller **memory** (long contexts, edge).

- ▶ **Time to first token (TTFT):** how long until the first output token appears; set by the prefill phase, which is compute-bound.
- ▶ **Time per output token (TPOT):** the gap between subsequent tokens; set by the decode phase, which is memory-bound.
- ▶ Request latency $\approx \text{TTFT} + \text{TPOT} \times \text{number of generated tokens}$.
- ▶ **Throughput:** total tokens per second across all requests. Batching raises throughput but can hurt per-request latency; serving systems balance the two.

- ▶ As covered earlier, **KV caching** keeps the key and value tensors of past positions in GPU memory so decode does not recompute them, at the price of memory that grows with layers, batch size, and context length.
- ▶ That memory is the binding constraint in serving: after the weights are loaded, whatever is left is what caps how many requests can run at once.
- ▶ Attention variants such as MQA and GQA shrink the cache by sharing key and value heads. The rest of this section attacks the same budget from two other directions: **how the cache is allocated** (paging) and **how many bits each entry costs** (quantization).



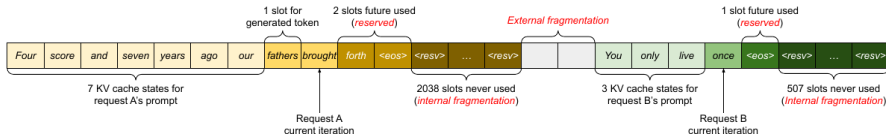
NVIDIA A100 40GB



Serving a 13B model on an NVIDIA A100 (40 GB): weights take 65% of memory, the KV cache over 30%. Kwon et al., [arXiv:2309.06180](https://arxiv.org/abs/2309.06180), Figure 1.

- ▶ A single decode stream is a matrix-vector workload that leaves most of the GPU idle; batching many requests turns it back into matrix-matrix work.
- ▶ **Static batching:** requests are grouped up front and the whole batch runs until its longest member finishes; short requests hold their slot while doing nothing.
- ▶ **Continuous batching:** scheduling happens per decode step; finished sequences leave the batch immediately and waiting requests join mid-flight. Introduced as **iteration-level scheduling** by Orca (Yu et al., 2022) and later popularised under the name continuous batching.
- ▶ Standard in modern serving stacks (vLLM, TensorRT-LLM, TGI): large throughput gains with no change to model outputs.

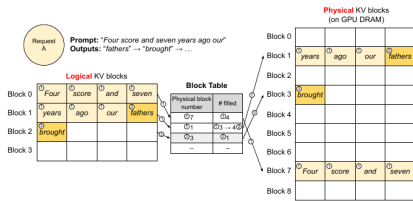
- ▶ Early serving systems stored each request's KV cache as one contiguous tensor, allocated up front for the maximum possible sequence length.
- ▶ Three kinds of waste follow: reserved slots not yet used, internal fragmentation (slots never used because the request ends early), and external fragmentation between buffers.
- ▶ Measured effective KV memory utilisation in such systems can drop to about 20%.



Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," [arXiv:2309.06180](https://arxiv.org/abs/2309.06180),

Figure 3.

- ▶ Borrow virtual memory from operating systems: split each request's KV cache into fixed-size blocks and allocate blocks on demand.
- ▶ A block table maps logical positions to physical blocks, so the cache no longer needs to be contiguous; waste is limited to the last partially filled block, and blocks can be shared across sequences (parallel sampling, shared prefixes).
- ▶ vLLM built on this achieves $2\text{--}4\times$ the throughput of prior serving systems at the same latency.



Block table translation. Kwon et al., [arXiv:2309.06180](https://arxiv.org/abs/2309.06180), Figure 6.

- ▶ Decode is memory-bound: for the large model, scoring K proposed tokens in one forward pass costs about the same as generating a single token.
- ▶ A small **draft model** proposes K tokens autoregressively; the large **target model** then evaluates all K positions in parallel.
- ▶ Accept draft tokens while they are consistent with the target distribution; at the first rejection, resample that token from an adjusted distribution and discard the rest.
- ▶ The output distribution is provably identical to decoding with the target model alone; the speed comes from accepting several tokens per target pass.
- ▶ Reported speedups with no change to the output distribution: $2\text{--}3\times$ on T5-XXL (Leviathan et al., 2023) and $2\text{--}2.5\times$ on Chinchilla 70B (Chen et al., 2023).

- ▶ Write q for the target model's next-token distribution and p for the draft model's. For each proposed token $\tilde{x}$, draw $r \sim U[0, 1]$ and **accept** it when

$$r < \min\left(1, \frac{q(\tilde{x} \mid x_{1:n})}{p(\tilde{x} \mid x_{1:n})}\right).$$

- ▶ So a token the target likes at least as much as the draft does is always accepted; one the draft over-proposed is accepted only in proportion q/p .
- ▶ On the first rejection, the remaining proposals are discarded and that position is resampled from the **residual** distribution $(q(x \mid x_{1:n}) - p(x \mid x_{1:n}))_+$, renormalised, where $(f)_+ = \max(0, f)$. This is what makes the accept/reject step exact rather than an approximation.
- ▶ If all K proposals are accepted, one extra token is sampled from the target's own distribution, so a single target pass can yield up to $K + 1$ tokens.

Modified rejection sampling, Algorithm 2 and Theorem 1 of Chen et al., "Accelerating Large Language Model Decoding with Speculative Sampling," [arXiv:2302.01318](https://arxiv.org/abs/2302.01318).

- ▶ **Kernel fusion & IO-aware attention** (e.g. FlashAttention-style): reduce HBM traffic.
- ▶ **Prefix caching**: reuse the KV cache of shared prompt prefixes (system prompts, few-shot examples) across requests.
- ▶ **Weight sharing / low-rank adapters (LoRA)**: smaller footprints where applicable.
- ▶ **Approximate attention** (long contexts): trade accuracy for sub-quadratic cost.

Problem:

- ▶ During **training**, a model does not have to operate in real time and does not necessarily face restrictions on computational resources. The primary goal is to extract as much structure from the given data as possible.
- ▶ But latency and resource consumption do become of concern if the model is to be deployed for **inference**.

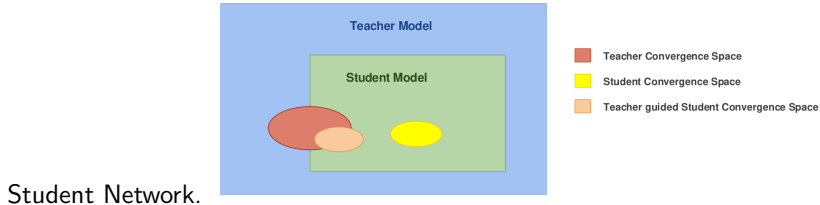
So what? We must develop ways to compress models for inference.

Idea:

- ▶ In 2006, Bucila et al. showed that it was possible to transfer knowledge from a large trained model (or ensemble of models) to a smaller model for deployment by **training it to mimic the larger model's output**.
- ▶ In 2014 Hinton et al. generalized the process and gave the name **Distillation**.

Main idea of distillation is that **training and inference are 2 different tasks**; thus a **different model should be used**.

Assumption: If we can achieve similar convergence using a **smaller network**, then the convergence space of the Teacher Network should **overlap** with the solution space of the



Teacher/Student convergence regions. Harvard AC215, L11.

What about this example?

This is a ...

$$\begin{pmatrix} 0 \\ 0 \\ 0.5 \\ 0.5 \end{pmatrix}$$



In the real world, the distinction is never that clear. Ideally, the **training labels must be soft**, to learn about the nuanced relations across the elements of the dataset.



The teacher shows the student the **relations** between the different classes, based on what it **learned** during its training stage.

From a dataset containing the original categories, the teacher outputs the **normalized probabilities** for each example.

	cow	car	dog	cat
Original labels	0	1	0	0
Probabilities	10^{-6}	0.9	0.1	10^{-9}

Adapted from:

Geoffrey Hinton, Oriol Vinyals & Jeff Dean, *Distilling the Knowledge in a Neural Network*

Distillation: Teacher – Student (Softening with Temperature)

To enhance the student's learning, the teacher **softens** its output z even more.

To do this, it introduces the softmax function with temperature T :

$$q_i = \frac{\exp(z_i / T)}{\sum_j \exp(z_j / T)}$$

The softened row is the same logits z pushed through this softmax at $T = 5$; like any softmax output, it still sums to 1.

“Soft” targets allow the student to learn the teacher's implicit knowledge about class similarities.

	cow	car	dog	cat
Original labels	0	1	0	0
Softmax probabilities	10^{-6}	0.9	0.1	10^{-9}
Softened, $T=5$	0.04	0.58	0.37	0.01

Adapted from:

Geoffrey Hinton, Oriol Vinyals & Jeff Dean,
Distilling the Knowledge in a Neural Network

A high value of T will dilute the information contained in the probabilities.

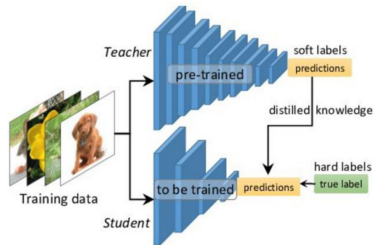
- ▶ The standard softmax is recovered for $T=1$.
- ▶ For $T>1$, the probabilities are softened. Typical values range between 2 and 5.
- ▶ For $T<1$, it approaches to the original one-hot labels.

	cow	car	dog	cat
Original labels	0	1	0	0
Softmax probabilities	10^{-6}	0.9	0.1	10^{-9}
Softened, $T=5$	0.04	0.58	0.37	0.01

Adapted from:

Geoffrey Hinton, Oriol Vinyals & Jeff Dean,
Distilling the Knowledge in a Neural Network

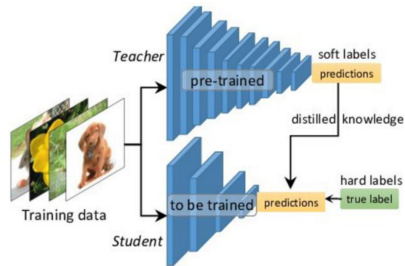
- ▶ The student is trained to both:
 - Solve the original task (matching the hard/true labels).
 - Replicate the softened probabilities (soft labels) from the teacher.
- ▶ This allows the student to learn not only the correct answer, but also the teacher's representation of class similarities.



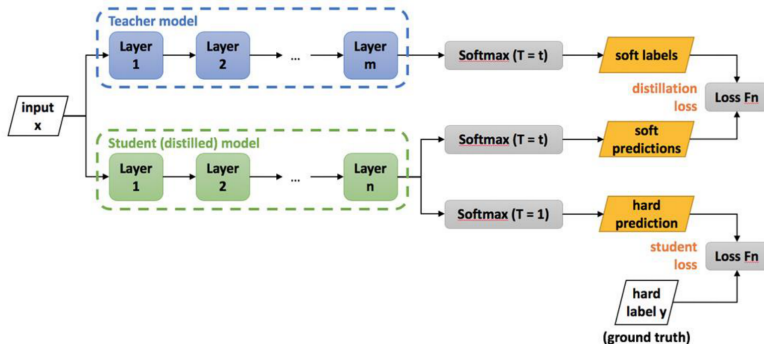
Geoffrey Hinton, Oriol Vinyals & Jeff Dean, *Distilling the Knowledge in a Neural Network*

It minimizes the sum of two different cross-entropy functions:

- ▶ One involving the original hard labels obtained using a softmax with $T = 1$.
- ▶ One involving the softened probabilities with $T > 1$.
- ▶ The gradients of the soft-target term scale as $1/T^2$, so in practice that term is multiplied by T^2 . Without it, raising T would quietly shrink the distillation signal relative to the hard-label one.



Geoffrey Hinton, Oriol Vinyals & Jeff Dean, *Distilling the Knowledge in a Neural Network*



Reference: Geoffrey Hinton, Oriol Vinyals & Jeff Dean, *Distilling the Knowledge in a Neural Network*.

$$L = \lambda L_{\text{student}} + (1 - \lambda) L_{\text{distillation}}$$

$$L_{\text{Distillation}} = - \sum_{k=1}^K p_T^k(x, T) \log(q_S^k(x, T))$$

In our example, for the example n in the dataset:

$$p_T(x, T) = [0.04, 0.58, 0.37, 0.01]$$

And our predictions

$$q_S(x, T) = [0.1, 0.45, 0.30, 0.15]$$

The resulting loss value for this example is:

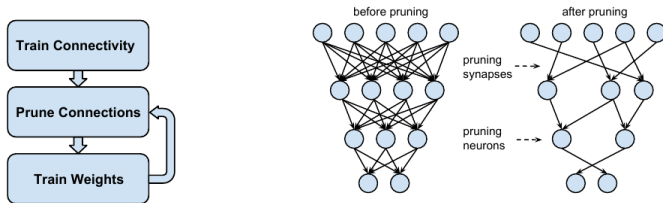
$$-\left[0.04 \log(0.1) + 0.58 \log(0.45) + 0.37 \log(0.30) + 0.01 \log(0.15)\right] \approx 1.02$$

- ▶ Classic distillation transfers the teacher's **output distribution**: the student matches softened logits, one prediction at a time.
- ▶ For reasoning models the valuable signal is the **trace**: the chain of intermediate steps the teacher produces before its answer.
- ▶ **Recipe**: generate many problems' worth of full reasoning traces with a large reasoning model, filter for correct answers, then fine-tune the small model on the traces with plain supervised learning. No logits, no RL. The filter-on-correct-answers step is the idea behind STaR (Zelikman et al., 2022).
- ▶ DeepSeek distilled R1 traces into Qwen and Llama models from 1.5B to 70B this way, using 800k curated samples and supervised fine-tuning only; the distilled students beat much larger non-reasoning baselines on math and code benchmarks.
- ▶ The trade: trace distillation is far cheaper than running RL on the small model, but the student inherits the teacher's reasoning style, including its mistakes.

DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,"

[arXiv:2501.12948](https://arxiv.org/abs/2501.12948).

- ▶ Trained networks are heavily over-parameterised: many weights contribute almost nothing to the output.
- ▶ **Magnitude pruning:** remove the connections with the smallest absolute weights, then retrain the survivors to recover accuracy; repeat iteratively.
- ▶ Han et al. (2015) cut AlexNet weights by $9\times$ and VGG-16 by $13\times$ with no accuracy loss.



Han et al., "Learning both Weights and Connections for Efficient Neural Networks," [arXiv:1506.02626](https://arxiv.org/abs/1506.02626), Figures 2–3.

- ▶ **Unstructured pruning** removes individual weights: highest compression, but the result is a sparse matrix that standard GPU kernels cannot run faster.
- ▶ **Structured pruning** removes whole neurons, attention heads, or layers: less aggressive, but leaves a smaller dense model that is faster on any hardware.
- ▶ **2:4 semi-structured sparsity**: keep 2 weights in every group of 4; NVIDIA tensor cores (Ampere onward) run it at up to $2\times$ math throughput (Mishra et al., 2021).
- ▶ For LLMs, one-shot post-training methods (SparseGPT, Wanda) reach about 50% sparsity with modest quality loss and no retraining.
- ▶ Pruning composes with distillation and quantization; production LLM serving still leans mostly on quantization, with pruning gaining ground through 2:4 hardware support.

Quantization is a **key** compression technique for **LLMs** and **large transformers** to fit on **constrained hardware**.

- ▶ **Goal:** run larger models on **constrained** hardware with acceptable accuracy.
- ▶ Lowering precision on **weights and/or activations** (e.g. FP32/FP16 $\rightarrow$ FP8) cuts **memory**, speeds **inference**, and reduces **energy**, at some accuracy cost; the right trade-off is use-case dependent.

- ▶ **Weights:** straightforward win on footprint; Llama 2 7B at FP16/BF16 $\approx$ **14 GB** $\rightarrow$ FP8 weights $\approx$ **7 GB** (blog numbers).
- ▶ **Activations:** intermediate layer outputs; quantizing compute can exploit **tensor cores** at lower bit width for more throughput.
- ▶ **KV cache** (decoder-only): grows with context, layers, and heads, often **several GB** at long context (e.g. 4096 tokens) alongside weights.
- ▶ **Deployment patterns:** weight-only INT8/INT4 for maximum weight memory savings (activations may stay higher precision); **KV cache quantization** when long context dominates.

- ▶ Bake quantization into training: forward/backward see **fake-quant** (quantize then dequantize) while master weights stay high precision for optimiser steps.
- ▶ Weights adapt to **rounding/clipping** error; fake-quant modules often **frozen** while weights are fine-tuned.
- ▶ One obstacle: $\text{round}(\cdot)$ has zero gradient almost everywhere, so a naive backward pass would stop learning. QAT uses the **straight-through estimator**: the rounding step is treated as the identity on the backward pass, passing the incoming gradient through unchanged.
- ▶ The result costs training compute that post-training methods do not, and is worth it mainly at the low bit-widths where post-training quantization starts to lose accuracy.

- ▶ **Weight-only quantization:** Quantize the model's trained weights to lower precision using a calculated scale (with or without a zero-point). Since weights are fixed after training, no input data is needed.
- ▶ **Weights and activations quantization:** Requires representative input data to measure activation statistics (by running the model), which are then used to determine quantization parameters (scale and zero-point) through a process called calibration.

- ▶ **Static quantization:** Calibration uses a dataset to determine quantization parameters once; these are kept fixed for all future inference.
- ▶ **Dynamic quantization:** Quantization parameters are computed on the fly during inference for each input, so no calibration set is needed but more runtime overhead is incurred.

- ▶ **Inference optimisation lowers latency, increases throughput, and reduces memory.**
- ▶ **KV caching** avoids recomputing past keys/values but grows memory with layers, batch, and length, motivating KV quantization, grouped attention, and paging.
- ▶ **Serving:** continuous batching keeps the GPU full; PagedAttention removes KV cache fragmentation; speculative decoding buys roughly 2–3× decode speed while preserving the target model's output distribution.
- ▶ **Distillation:** train a compact **student** to mimic a **teacher** for deployment; **soft targets** at raised **temperature** carry knowledge beyond one-hot labels.
- ▶ **Pruning:** remove low-value weights (unstructured, structured, or 2:4 semi-structured) and retrain; composes with the other compression techniques.
- ▶ **Quantization:** shrink **weights**, **activations**, and the **KV cache** via lower-bit formats.

1. Vaswani et al., “Attention Is All You Need,” NeurIPS 2017. [arXiv:1706.03762](#)
2. Bucila, Caruana, Niculescu-Mizil, “Model Compression,” KDD 2006, pp. 535–541. [PDF](#)
3. Hinton, Vinyals, Dean, “Distilling the Knowledge in a Neural Network,” NIPS 2014 Deep Learning Workshop. [arXiv:1503.02531](#)
4. Sanh, Debut, Chaumond, Wolf, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” 5th Workshop on Energy Efficient Machine Learning and Cognitive Computing, NeurIPS 2019. [arXiv:1910.01108](#)
5. Zelikman et al., “STaR: Bootstrapping Reasoning With Reasoning,” NeurIPS 2022. [arXiv:2203.14465](#)
6. DeepSeek-AI, “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,” 2025. [arXiv:2501.12948](#)
7. Jacob et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” CVPR 2018. [arXiv:1712.05877](#)
8. Leviathan, Kalman, Matias, “Fast Inference from Transformers via Speculative Decoding,” ICML 2023. [arXiv:2211.17192](#)
9. Chen et al., “Accelerating Large Language Model Decoding with Speculative Sampling,” 2023. [arXiv:2302.01318](#)

10. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” SOSP 2023. [arXiv:2309.06180](#)
11. Yu et al., “Orca: A Distributed Serving System for Transformer-Based Generative Models,” OSDI 2022. [Paper](#) (iteration-level scheduling)
12. Han et al., “Learning both Weights and Connections for Efficient Neural Networks,” NeurIPS 2015. [arXiv:1506.02626](#)
13. Mishra et al., “Accelerating Sparse Deep Neural Networks,” 2021. [arXiv:2104.08378](#) (2:4 sparsity)
14. Frantar & Alistarh, “SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot,” ICML 2023. [arXiv:2301.00774](#)
15. Sun et al., “A Simple and Effective Pruning Approach for Large Language Models,” ICLR 2024 (Wanda). [arXiv:2306.11695](#)
16. Verma, Vaidya, “Mastering LLM Techniques: Inference Optimization,” NVIDIA Developer Blog, 17 Nov. 2023. [Article](#)
17. Wang, Spindler, “Model Quantization: Concepts, Methods, and Why It Matters,” NVIDIA Developer Blog, 24 Nov. 2025. [Article](#)

20. Zandieh, Daliri, Han, “QJL: 1-Bit Quantized JL Transform for KV Cache Quantization with Zero Overhead,” AAAI 2025. [arXiv:2406.03482](#)
21. Han, Kacham, Karbasi, Mirrokni, Zandieh, “PolarQuant: Vector Quantization with Polar Transformation,” AISTATS 2026. [arXiv:2502.02617](#)
22. Gao & Long, “RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search,” SIGMOD 2024. [arXiv:2405.12497](#)
23. M. Naik, *Introduction*, Lecture 1, CIS 7000: *Large Language Models*, University of Pennsylvania, Fall 2024. [PDF](#)
24. Harvard University IACS, AC215: *MLOps, LLMops & AIOps: Productionizing AI Systems* (Fall 2024), Lecture 11: *Models Compression Techniques* (slides 7–34 used for the distillation figures). [PDF](#)
25. T. Chen & Z. Jia, *LLM Finetuning Techniques*, Lecture 17, CMU 15-779: *Advanced Topics in Machine Learning Systems (LLM Edition)*, Fall 2025. [PDF](#)